

AQI-Pearls-Predictor

Serverless Machine Learning Pipeline for Multi-Horizon Air Quality Forecasting in Lahore

Project Report

Developed by: Abu Hurraira

Email: abuhurraira1904@gmail.com

Internship Project - Pearls AQI Predictor

September 2026

1. Introduction

Pearls AQI Predictor is an end-to-end, fully serverless machine learning system that forecasts Lahore's Air Quality Index (AQI) three days into the future, at 24-hour, 48-hour, and 72-hour horizons. The project was undertaken as part of an internship, following a brief that called for a scalable, automated pipeline covering data collection, feature engineering, model training, explainability, and an interactive dashboard.

The system was built to run without any dedicated server: an hourly feature pipeline and a daily training pipeline both run on GitHub Actions, feature data is stored in a lightweight file-based feature store, trained models are versioned in a model registry, and predictions are served through a publicly deployed Streamlit dashboard.

1.1 Objectives

- Build an hourly feature pipeline that fetches, validates, and aggregates live air quality data for Lahore.
- Engineer a robust, reusable feature set and store it in a feature store suitable for both training and serving.
- Train and compare multiple candidate models per forecast horizon, with automated champion/challenger promotion.
- Provide model explainability so predictions are not a black box.
- Automate the entire pipeline end-to-end with no manual intervention.
- Present the results through a live, interactive dashboard with hazard-level alerting.

2. System Architecture

The system is organized into four stages that run independently of one another, connected through a shared feature store:

- Hourly Feature Pipeline - fetches raw station data, validates it, aggregates it into a single city-wide reading, and writes it to the feature store.
- Feature Store - a Feast-based offline store holding the full historical feature history.
- Daily Training Pipeline - reads the feature history, prepares training data, trains and evaluates candidate models per horizon, and promotes the better model into the registry.
- Streamlit Dashboard - reads the latest features and the current champion models to serve live predictions, historical trends, EDA, and explainability views.

Scheduling for both the hourly and daily pipelines is handled through GitHub Actions workflows, externally triggered via cron-job.org (see Section 7 for why an external trigger was needed).

3. Data Collection and Source Selection

Live AQI, pollutant, and weather readings are pulled from the AQICN API across a curated set of Lahore monitoring stations. A significant part of the early project work involved evaluating which station networks to trust and combine.

3.1 Comparing Station Networks

Four station networks were available for Lahore through AQICN: the Pakistan Punjab Air Quality Network (PPAQN), The Urban Unit, Citizen Science stations, and Clarity. Readings across these networks were compared, and discrepancies between them were noticeable enough to require a deliberate choice rather than blending all four indiscriminately.

The US Embassy station, often treated as a reference source in other cities, was found to be returning stale data dated back to 2025 and was excluded outright rather than risk contaminating the feature set with outdated readings.

PPAQN and The Urban Unit were selected as the two trusted sources. Citizen Science and Clarity stations were not used in the final aggregation.

3.2 Primary and Secondary Source Selection

Between the two selected networks, The Urban Unit was designated the primary source and PPAQN the secondary source, for two reasons:

- Update frequency: The Urban Unit refreshes roughly every 15 minutes, while PPAQN updates only every 2–3 hours. More frequent updates make Urban Unit readings far more likely to be current at the time of any given hourly fetch.
- Spatial coverage: The Urban Unit's stations are more evenly spread across Lahore, giving a more representative citywide picture than PPAQN alone.

This distinction also directly shaped the staleness-threshold design described in Section 3.4, since a network updating every 15 minutes should be held to a stricter staleness bar than one updating every few hours.

3.3 Station Discovery Workaround

The AQICN map-bounds endpoint, which would normally be used to programmatically discover all stations within a geographic bounding box, did not reliably return Urban Unit stations. Rather than depend on an unreliable discovery mechanism, the relevant station IDs were curated manually and written to a static configuration file (`AQI_Stations.json`), which the fetch pipeline reads from directly. This traded a small amount of manual upkeep for a dependable, reproducible list of stations.

3.4 Staleness Detection

Each station's most recent reading is checked against a maximum allowed age before being treated as valid. Different networks were assigned different thresholds to reflect their expected update cadence - a tight threshold for The Urban Unit (which updates every ~15 minutes) and a looser one for PPAQN

(which updates every 2–3 hours), with a default fallback threshold applied to any other source. An absolute time-based threshold was chosen over a stateful "last seen" comparison for simplicity, since GitHub Actions runners do not persist state between runs (see Section 6.2).

3.5 Outlier Removal

Station readings that deviate from the citywide median AQI by more than 50 points are flagged and excluded from the aggregation step, on the assumption that such large deviations are more likely to reflect sensor error than genuine hyper-local pollution spikes.

3.6 Filling Missing Weather Fields

PPAQN does not report temperature or humidity values. Rather than leave these fields null for PPAQN-only readings, a median temperature and humidity value is taken from The Urban Unit's stations and applied city-wide. This was considered an acceptable simplification because temperature and humidity vary far more uniformly across a city than AQI itself, which is comparatively hyper-local due to traffic, industry, and localized emissions sources.

4. Feature Engineering

4.1 City Centroid Calculation

A one-time script computes Lahore's geographic centroid by averaging the latitude and longitude of all valid stations, and persists the result to a `city_centroid.ini` configuration file. This avoids recomputing the centroid on every pipeline run and gives a stable reference point for distance-based weighting.

4.2 Inverse Distance Weighting (IDW)

Station-level pollutant readings are combined into a single city-wide value using inverse distance weighting relative to the city centroid: stations closer to the centroid contribute proportionally more to the aggregated value than distant ones. If a station happens to sit essentially at the centroid, its reading is used directly rather than diluted through weighting.

A grid-based inverse-distance-weighted interpolation approach - producing a full spatial AQI surface across Lahore rather than a single citywide number - was considered. It was ultimately rejected as being out of scope: the project's forecasting target is a single city-level AQI value, not a spatial map, and a grid-based approach would have added considerable complexity without serving that goal.

4.3 Distance Calculation

The Haversine formula was used to compute distances between each station and the city centroid, rather than a flat Euclidean approximation, because it accounts for the curvature of the Earth and gives more accurate real-world distances even at the relatively small scale of a single city.

4.4 Derived Time and Trend Features

In addition to raw pollutant and weather values, the feature set includes hour-of-day, day-of-week, and month, along with an `aqi_change_rate` feature capturing the rate of AQI change since the previous reading (in AQI points per hour). Rows with no valid predecessor, or with a gap exceeding the allowed threshold, have this rate recomputed as null rather than a misleading estimate.

5. Feature Store

5.1 Initial Approach: Hopsworks

The feature store was originally implemented on Hopsworks, using HUDI as the storage format. HUDI was chosen over Delta Lake specifically because it was the simpler option and did not require pulling in an additional dependency, which mattered given the project's serverless, lightweight constraints.

This approach was abandoned after Hopsworks' free-tier offline feature store repeatedly became stuck during the Spark materialization initialization stage (see Section 7.2). This was a platform-side reliability issue outside the project's control.

5.2 Migration to Feast

Feast was adopted as a replacement offline feature store. Only the local, file-based offline store was used - no online store was needed, since the project's use case (daily batch training and periodic dashboard reads) does not require low-latency online feature serving. The Hopsworks feature-store logic in the pipeline was gradually swapped out for equivalent Feast calls, while Hopsworks itself was retained purely as the model registry.

Feast's offline store persists the full feature history to a local Parquet file, which is committed back into the GitHub repository itself. This was necessary because GitHub Actions runners are ephemeral and do not retain any state between runs - committing the Parquet file back to the repo gives the pipeline a durable, versioned source of truth that survives across runs.

5.3 Timestamp and Timezone Handling

Timestamps required careful normalization throughout the pipeline. Feast's `FileSource` requires a genuine `datetime64[ns, UTC]` column on write, which then needs to be converted back to epoch-millisecond integers on read to match the format expected by downstream cleaning code. Separately, the raw AQICN API returns timestamps in ISO 8601 format with an explicit timezone offset, which needed to be reconciled consistently with the UTC convention used internally, to avoid subtle off-by-several-hours errors creeping into hour-of-day features or staleness checks.

These timezone and representation mismatches were the root cause of three distinct bugs encountered later in the project (detailed in Section 9.1), and were ultimately consolidated into a single, well-tested normalization step rather than being handled piecemeal at each call site.

6. Automation and Scheduling

Both the hourly feature pipeline and the daily training pipeline are implemented as GitHub Actions workflows, triggered via `workflow_dispatch`.

6.1 GitHub Actions Scheduling Delays

GitHub Actions' built-in cron scheduling proved unreliable in practice, with runs frequently delayed beyond their scheduled trigger time - a known platform-level limitation of GitHub's shared runner queue during periods of high demand, and not something the pipeline design could fix directly. The workaround was to trigger the workflows externally via `cron-job.org`, which calls the GitHub Actions `workflow_dispatch` API on a reliable schedule instead of depending on GitHub's own cron trigger.

6.2 GitHub Actions Runner Statelessness

Each GitHub Actions run starts from a clean environment with no memory of previous runs. This shaped two design decisions described elsewhere in this report: the decision to commit the Feast Parquet file back into the repository as durable storage (Section 5.2), and the decision to use absolute time-based staleness thresholds rather than stateful "last seen" tracking (Section 3.4).

6.3 Repository Write Permissions

Committing the updated Feast Parquet file back to the repository from within a GitHub Actions run required explicitly enabling write permissions for the workflow, under Settings → Actions → General → Workflow permissions → Read and write permissions. This is a one-time repository configuration step, not something set in the workflow YAML itself.

7. Training Pipeline

7.1 Deduplication and Hourly Bucketing

Because station data can be fetched at slightly irregular times, rows are grouped into hour-of-the-absolute-day buckets, and for each bucket the reading closest to the top of the hour is kept as that hour's representative row, within a 45-minute matching tolerance. This removes duplicate or near-duplicate entries for the same hour while keeping the most representative reading.

7.2 Target Construction

Training targets for each horizon are built by shifting the cleaned, deduplicated feature rows forward in time: for a given row, the 24h/48h/72h target is the AQI value recorded closest to that many hours later, matched using a nearest-time join within a tolerance window. Rows with no matching future reading within tolerance are left without a target for that horizon and excluded from training for that horizon specifically.

7.3 Per-Horizon Model Training

A separate model is trained independently for each of the three horizons, since the relationship between current conditions and AQI 24 hours out is not necessarily the same shape as the relationship 72 hours

out. For each horizon, two candidate model families from the scikit-learn family are trained and compared: Ridge Regression and Random Forest.

Data for each horizon is split chronologically into training (70%), validation (15%), and test (15%) sets, respecting the time-ordered nature of the data rather than shuffling randomly. Each candidate is trained on the training set and evaluated on the validation set; the candidate with the lowest validation RMSE is selected as that horizon's winner and then evaluated once more on the held-out test set for final reporting.

7.4 Random Forest Size Constraint

An unconstrained Random Forest produced a serialized model of roughly 119MB, which caused truncated downloads when registering and later retrieving the model from Hopsworks. The Random Forest was constrained (`max_depth=15`, `min_samples_leaf=5`) specifically to keep the serialized model size manageable for reliable registry storage and retrieval, at an acceptable cost to model complexity.

7.5 Champion/Challenger Promotion

After a new challenger model is trained for a horizon, it is compared against the current champion model already registered in the Hopsworks Model Registry for that horizon. The champion is re-evaluated (re-scored) on the challenger's own freshly generated test split, rather than relying on the champion's originally recorded metrics, to ensure the comparison is fair and reflects both models against the same data.

The challenger replaces the champion only if it improves test RMSE by more than 1% relative to the champion's re-scored RMSE. A fixed absolute improvement threshold of 0.0 was tried initially, but this allowed near-duplicate models to be promoted purely from floating-point noise; a percentage-based threshold was adopted instead since it scales sensibly regardless of a horizon's typical RMSE magnitude.

Because each promotion battle is fought using whatever new data has accumulated since the last training run, it is worth noting that the metrics recorded in the Hopsworks registry for a given champion reflect the data available at the time that model was promoted - not the current day's data. Registry metrics should therefore be read as a historical snapshot rather than a live, continuously updated score.

7.6 Handling Insufficient Data

If a horizon does not yet have enough matched (feature, target) rows to train reliably, training for that horizon is skipped entirely for that run rather than proceeding with an unreliable model. This matters early in the project's data-collection timeline, before enough historical depth had accumulated for the longer 72-hour horizon in particular.

8. Explainability

Two complementary explainability techniques are used so that model predictions are not a black box, and so that every horizon has some form of explanation available regardless of which model family won that horizon.

8.1 SHAP

SHAP's TreeExplainer is applied to horizons where Random Forest is the winning model. This produces a global summary (beeswarm) plot showing overall feature importance and direction of effect, along with local waterfall plots for the best- and worst-predicted test rows. TreeExplainer is tree-model-specific, so horizons where Ridge Regression wins are skipped for SHAP, with a logged warning rather than a silent gap.

8.2 LIME

LIME was added specifically to cover the gap left by SHAP: because LIME is model-agnostic, it works identically regardless of whether a horizon's winner is Ridge or Random Forest. LIME is fundamentally a local, per-prediction method, so its "summary" view in this project is an approximation built by averaging local feature-weight explanations across a sample of test rows, rather than a true global measure like SHAP's beeswarm plot.

9. Challenges, Debugging, and Resolutions

9.1 Timestamp Normalization

Timestamps arrived in three different shapes at different points in the pipeline: proper `datetime64` columns (timezone-aware or naive), plain numeric epoch-millisecond integers, and - in the trickiest case - a mixed object-dtype column. The mixed case occurred when a `datetime64` frame and an integer frame were concatenated before normalization: pandas does not coerce the result back into a single consistent dtype, so the column degrades into literal mixed Timestamp and int objects. This, combined with the ISO timestamp timezone offsets from the raw API, caused three separate bugs traced to three different causes: automatic date parsing during JSON reads, an explicit but incorrect `pd.to_datetime` call, and Hopsworks' own `feature_group.read()` returning a true `datetime64` column because the feature store schema defines timestamp as a `TIMESTAMP` type.

The fix was to write a single `normalize_timestamp_column()` function that detects and correctly handles all three shapes, and to call it once, first, at the very start of the cleaning stage - rather than re-implementing timestamp handling at each individual call site every time a new data source introduced yet another representation.

9.2 GitHub Actions Scheduling Delays

As covered in Section 6.1, GitHub Actions' scheduled runs were unpredictably delayed, a platform-side limitation outside the project's control. Resolved by switching the scheduling trigger to `cron-job.org`.

9.3 Hopsworks Spark Materialization Failures

The Hopsworks free-tier offline feature store repeatedly became stuck during the Spark job initialization stage of feature materialization (job type `offline_fg_materialization`, error 120003), affecting all feature groups project-wide rather than being specific to this project's data or configuration. As this was a platform-side issue with no available workaround on the free tier, the project migrated its feature-storage layer to Feast (Section 5.2), keeping Hopsworks solely for model registry duties, which did not exhibit the same failure.

9.4 Promotion Fairness and Threshold Sensitivity

Two related issues surfaced in the champion/challenger promotion logic: comparing a challenger against the champion's stale, originally recorded metrics rather than a fresh evaluation, and using an absolute (rather than relative) RMSE improvement threshold that let near-duplicate models get promoted on floating-point noise. Both were resolved together (Section 7.5): re-scoring the champion on the challenger's fresh test split, and switching to a 1% relative RMSE improvement threshold.

9.5 Oversized Random Forest Models

An unconstrained Random Forest serialized to approximately 119MB, causing truncated downloads from the Hopsworks Model Registry. Resolved by constraining `max_depth` and `min_samples_leaf` (Section 7.4).

9.6 Dashboard Dependency Conflicts

Deploying the Streamlit dashboard surfaced a protobuf version conflict between streamlit, hopsworks, and feast. This was resolved by pinning `streamlit==1.37.1`, removing an unnecessary `pywinpty` dependency, and adjusting image-display calls (`use_column_width=True`) for compatibility with that pinned version.

10. Exploratory Data Analysis - Summary of Findings

Full EDA logic lives in a shared `eda_plots.py` module (rather than being written directly inside the notebook), specifically so that the same plotting functions could later be reused by the dashboard's EDA tab without duplicating code. The following summary is drawn from the `AQI_EDA.ipynb` notebook's own findings section.

Row Count and History Span

Of the rows collected, roughly 180 were unique hourly readings out of 196 total rows, reflecting some duplicate-hour entries removed during cleaning. On average, Lahore's AQI sat around 114 - already at an unhealthy level for sensitive groups - with a standard deviation of about 25, meaning most readings stayed within the moderate-to-unhealthy band. The maximum observed AQI over the collection window reached 186, unhealthy even for the general population, though not yet in the hazardous range.

Diurnal, Weekly, and Monthly Patterns

A clear dip in AQI was observed around midday, consistent with increased sunlight and wind dispersing pollutants, with a rise overnight likely driven by increased evening traffic and a reduced atmospheric boundary layer height trapping emissions closer to the surface. AQI also trended higher on Thursdays and Fridays and lower on Mondays and Sundays, plausibly tied to weekly traffic activity patterns. Monthly patterns could not yet be meaningfully assessed, since the collection window did not yet span multiple months.

Distribution Shape

The AQI distribution is right-skewed, consistent with values clustering at generally poor-but-not-extreme levels with occasional spikes upward - the expected shape given AQI's lower bound at zero. PM2.5, the dominant pollutant in Lahore, showed a similar distribution shape to AQI overall, while PM10 was also right-skewed, in line with expectations.

Correlation Highlights

PM2.5 and PM10 both showed strong correlation with AQI, unsurprising given that AQI is derived directly from pollutant concentrations. Beyond the pollutants themselves, humidity showed the next-strongest relationship with AQI, followed by temperature.

Data Quality Observations

Some gaps in the historical record were traced to GitHub Actions scheduling issues and Hopsworks materialization failures (Sections 9.2–9.3), along with one larger gap corresponding to the migration from Hopsworks to Feast as the feature store.

Train/Validation/Test Variance and Model R^2

The chronological train split showed higher AQI variance than the validation and test splits, largely due to the OpenAQ historical backfill data included in the earlier portion of the timeline. This variance mismatch between splits - despite being a necessary trade-off to obtain enough historical training data - directly explains the comparatively low test R^2 scores observed during model evaluation, reinforcing that RMSE (Section 7.3) rather than R^2 is the more meaningful metric for this project's hazard-alerting goal.

11. Dashboard

The dashboard is built with Streamlit and deployed publicly on Streamlit Community Cloud, refreshing automatically every 90 minutes. It is organized into tabs covering live predictions, historical trends, exploratory data analysis, and explainability (both SHAP and LIME), with hazard-level color coding applied consistently throughout using the same AQI band definitions used elsewhere in the pipeline. EDA and explainability plots are served as pre-generated static images by default, with a manual "Regenerate" action available per tab, rather than recomputing every plot on every page load - a deliberate trade-off favoring dashboard responsiveness over always-fresh visuals.

12. Summary of Key Design Decisions

Decision Area	Choice Made	Reasoning
Feature store	Feast (local offline, file-based)	Hopsworks free tier was unreliable for materialization; Feast's local store required no online-serving complexity the project didn't need.
Model registry	Hopsworks	Registry functionality was stable even when the feature-store side was not.
Storage format (early)	HUDI over Delta Lake	Simpler, no added dependency, sufficient for project needs.
Trusted data sources	Urban Unit (primary) + PPAQN (secondary)	Better update frequency and spatial spread than alternatives; Citizen Science, Clarity, and the stale US Embassy station excluded.
Spatial aggregation	Single citywide IDW value	A grid-based spatial surface was out of scope; the target is one city-level AQI figure.
Distance metric	Haversine	Accounts for Earth's curvature; more accurate than flat Euclidean distance.
Train/test split	Chronological 70/15/15	Respects time-series autocorrelation; avoids leakage from random shuffling.
Headline metric	RMSE over R^2	Aligned with hazardous-threshold alerting use case; low R^2 on a short window is expected, not a failure.
Promotion threshold	1% relative RMSE improvement	Absolute threshold of 0.0 allowed noise-level promotions; relative threshold scales properly across horizons.
Promotion fairness	Champion re-scored on challenger's test split	Prevents comparison against stale, outdated champion metrics.
Random Forest size	max_depth=15, min_samples_leaf=5	Unconstrained RF produced ~119MB models, causing truncated downloads.
Explainability	SHAP (RF only) + LIME (all models)	SHAP's TreeExplainer covers RF winners; LIME fills the gap for Ridge-winning horizons.
Dashboard rendering	Static PNGs + manual regenerate	Avoids recomputing expensive plots on every page load.
Scheduling trigger	cron-job.org instead of native GitHub Actions cron	GitHub Actions' own scheduler introduced unpredictable delays.

13. Current State

13.1 Current State

- Hourly feature pipeline: complete, running end-to-end in GitHub Actions.
- Feature store: fully migrated to Feast, with Parquet committed to the repository as durable storage.

- Training pipeline: complete, including per-horizon model comparison and champion/challenger promotion.
- Explainability: complete for both SHAP and LIME, covering all horizons regardless of winning model.
- EDA: complete, with reusable plotting functions shared between the notebook and dashboard.
- Dashboard: complete and deployed publicly, covering predictions, trends, EDA, and explainability.

14. Conclusion

This project delivered a complete, serverless, end-to-end AQI forecasting system for Lahore - from raw multi-source data collection through to a public, explainable dashboard - while navigating real platform-level constraints (unreliable free-tier feature-store materialization, inconsistent scheduler timing, inconsistent multi-source station data) with pragmatic, well-reasoned workarounds rather than deferring to a more complex or costly infrastructure. The resulting pipeline is fully automated, reproducible, and designed so that its modular pieces (feature engineering, training, explainability, and dashboard logic) can be reused, extended, or restructured independently.